

3D interaktív környezet/játék fejlesztése Gaussian splatting alapon

Önálló laboratórium 1.

Utolsó módosítás: 2025. 05. 26.

Tartalomjegyzék

Általános információk	2
Projekt célja	2
Témakiírás	2
Keletkezés körülményei	2
Bevezetés	3
Motiváció	3
Megoldás felvázolása	3
Dokumentum struktúrája	3
Kapcsolódó munkák	4
Cikk	4
Gyűjtemény	4
Mesh2splat	4
Részegységek	5
Egyszerre több pontfelhő	5
Pontfelhők transzformációja	5
Automata objektum-megtalálás	7
Mélységhelyes összekombinálás	8
Projektek kombinálása	8
Adatok beolvasása	9
Eredmények, összefoglalás	10
Fejlesztési lehetőségek	10

Általános információk

Projekt célja

A projekt célja Gaussian splatting alapú modellek integrálása a Godot játékmotorba. A félév végére lehetséges több splat-es objektumot megjeleníteni hagyományos objektumok mellett, és ezek között interakciókat létrehozni.

Témakiírás

[A tanszéki témakiírás.](#)

Napjaink egyik úttörőnek számító, igen aktív kutatási területe az ún. Gaussian splatting, ahol egy 3D környezetet Gauss-eloszlású, ellipszoid-alakú "splat"-ek halmazaként adunk meg. A 3D modellezésben használt klasszikus reprezentációkkal (pl. poligonhálókkal, vagy pontfelhőkkel) összehasonlítva a Gaussian splatting számos előnnyel bír, például:

- Pusztán RGB képek alapján rekonstruálható vagy a képekhez hasonlóan MI technikákkal automatikusan generálható is.
- Fotorealistikus minőségű rekonstrukciót lehet általa produkálni, akár komplex, nézeti iránytól függő megjelenéssel (pl. csillanások, tükröződések, részletes textúrák).
- Renderelése GPU-n keresztül - a háromszögek raszterezéséhez hasonló módon - igen hatékonyan (100+ FPS-el) elvégezhető.

A Gaussian splattinghez kapcsolódó kutatások és fejlesztések nagy része ezidáig a 3D rekonstrukciós, generatív AI, valamint robotikai alkalmazásokra fókuszált, azonban felmerül a lehetőség, hogy komplett számítógépes játékot, vagy más interaktív (akár VR/AR) környezetet is létrehozzunk Gaussian Splatting segítségével. Ennek kapcsán a Gaussian splatting rekonstrukció és/vagy AI-alapú generálás problémáin felül számos megoldandó feladat elképzelhető:

- Gaussian splatting renderelés integrációja grafikai engine-be (pl. Unity/Unreal vagy saját engine).
- Gaussian splatting modellek animációja / fizikai szimulációja.
- Gaussian splatting integrációja AR/VR interfésszel.
- Gaussian splatting rekonstrukciók szegmentálása statikus környezetre, dinamikus objektumokra, stb.
- Stb...

Keletkezés körülményei

Ezt a programot a BME mérnökinformatikus MSc képzés Önálló laboratórium 1. tárgyának keretében készítem.

A program több projekten alapszik:

- A [Godot](#) nevű játékmotor adja a projekt keretét
- A [GodotGaussianSplatting](#) egy Gaussian splat-os objektumokat megjelenítő program Godot-tal megvalósítva, ez adja a megjelenítést
- A [VitaVehicle](#) egy félig-meddig realiztikus fizikai alapú autó szimulátor, ez adja a fizikát

A hozzáadott forráskód, ugyanúgy, mint a fentebb felsorolt projektek is, MIT licenz alatt elérhetők a [Githubon](#).

Bevezetés

Motiváció

Gaussian splatting modelleket létre lehet hozni valós objektumokról, könnyebben, mint háromszögháló alapú modelleket, és valóságosabban is ki tudnak nézni. Ezek integrálása egy játékmotorba lehetővé teszi, hogy akár részben ilyen modellekkel készítsünk játékokat. Ez azért is lehet előnyös, mert például egy plüssjáték Gaussian splatting modellként elég élethűen néz ki, míg tradicionális modellezéssel jóval nehezebb ilyen részletességű modellt létrehozni.

Megoldás felvázolása

Adott a megtekintő program, amely be tud tölteni egy `.ply` kiterjesztésű splat-os modellt, és meg tudja azt jeleníteni a játékmotor keretein belül. Illetve adott példaként az autó-szimuláló program. Adottak továbbá splat-os modellek, amiket lehet használni.

Ennek megfelelően a fél éves munka a következő részekre osztható:

- Megjeleníteni egyszerre több pontfelhőt
- Ezeket a pontfelhőket függetlenül mozgatni
- Feltölteni pontfelhőnként transzformációs mátrixokat, hogy tetszőlegesen mozoghassanak a modellek
- Ezeket a transzformációkat megfelelően feltölteni minden képkockában, lehetőleg automatizált módon megtalálva a hozzájuk tartozó játékobjektumot, ahonnan a transzformáció származik
- Mélység helyesen összekombinálni a pontfelhős, és a sima modelleket
- Összevonni a két projektet (gsplat-os, illetve az autós)
- Robusztusabbá tenni a `.ply` fájl-beolvasást, hogy a `.splat`-ból [SuperSplat](#)-tal konvertált `.ply` fájlokat is képes legyen beolvasni
- Javítani felmerülő problémákat
 - Stuttering: camera és transzformációs mátrix feltöltésének a process priority állítása lett a megoldás
 - Artifacts mélység: a shader kód párhuzamosan tölt be adatokat, és dolgozza fel őket, ehhez viszont szükségeses shared bufferek használata

Dokumentum struktúrája

Fentebb található a keletkezési körülmények, az alapul vett repository-k, és a fél év során végzett programozási munkák listája. Lejjebb találhatóak további kapcsolódó munkák, az előbbi lista részletesebb kifejtése, majd az összefoglalás, és a jövőbeni lehetséges továbbfejlesztések listája.

Kapcsolódó munkák

Cikk

Kerbl et al.: 3D Gaussian Splatting for Real-Time Radiance Field Rendering: [itt](#). Ez a papír vezette be a Gaussian splat-ok használatát, ezzel lehetővé téve a valós idejű megjelenítést.

Gyűjtemény

[Ezen](#) a linken található sok kapcsolódó cikk és szoftver.

[Ez a projekt](#) fizikával is egybeköti a splatokat.

A téma újabban felkapott lett, sok egyéb projektet lehet találni a témában.

Mesh2splat

[Ez](#) a projekt az Electronic Arts-é, és sima objektumokat hivatott átkonvertálni pontfelhőkké. Az önlab keretében ezt ki is próbáltam, ehhez portoltam a szoftvert linuxra, és a szükséges módosításokat a kérésükre egy [pull request](#)ként fel is töltöttem. Ezt ezidáig nem mergelték még, de többek aktivitásából látszik, hogy érdeklődnek a módosítások iránt.

Végül az eredményező splat fájlokat nem tudtam felhasználni, mivel azok újralightolható formájúak, ami jó lenne egy játékhoz, viszont a megjelenítő ezt nem támogatja jelenleg.

Részegységek

Itt felsorolom a végső projekt létrejöttéhez szükséges részegységeket, lépéseket.

Egyszerre több pontfelhő

A megjelenítő egy pontfelhő megjelenítésére volt felkészítve, nekünk viszont egy játékban jórészt több pontfelhőt is célszerű megjeleníteni: például az autó teste és a 4 kereke már eleve 5 külön objektum. Ehhez azt találtam ki, hogy a CPU-n összeuniozom a pontfelhőket, az adataikat, viszont elmentem azt is, hogy mely indexeknél vannak a határok. Ezután a GPU-ra feltöltéskor kihasználok egy float-ot, ami eddig padding célt szolgált, és minden feltöltött splat esetén egy ID-t is továbbítok, ami megmondja, hogy hányadik objektumhoz tartozik.

```
class_name PlyFile extends Resource

var size : int
var vertices : PackedFloat32Array
var properties : Array[StringName]
var split : Array[int] # indices where new objects start

...

static func merge(pc1 : PlyFile, pc2 : PlyFile) -> PlyFile:
    var merged := PlyFile.new()
    merged.size = pc1.size + pc2.size
    assert(pc1.properties.hash() == pc2.properties.hash())
    merged.properties = pc1.properties
    merged.vertices = PackedFloat32Array(pc1.vertices)
    merged.vertices.append_array(pc2.vertices)
    merged.split.append_array(pc1.split)
    merged.split.append(pc1.vertices.size())
    for s in pc2.split:
        merged.split.append(s + pc1.vertices.size())
    return merged

...

static func load_gaussian_splats(point_cloud : PlyFile, stride : int,
    device : RenderingDevice, buffer : RID, should_terminate_reference : Array[bool],
    num_points_loaded : Array[int], callback : Callable):
    ...

        ### Opacity
        points[b+6+4] = 1.0 / (1.0 + exp(-p[v+54]))

        ### ID for differentiating between objects
        points[b+11] = 0
        for k in point_cloud.split:
            if v >= k:
                points[b+11] += 1

    ...
```

Pontfelhők transzformációja

Miután a GPU-n már lehet tudni, hogy melyik splat melyik objektumhoz tartozik, így lehetséges objektumonként más transzformációt alkalmazni a splatokra. Ehhez létre kellett hozni, és fel kell tölteni egy új uniform buffert:

```
const MAX_OBJECT_COUNT := 16 # number of gsplat object transforms, same as in gsplat_projection.gsl
var object_transforms : Array[Transform3D]

...
```

```

descriptors['transforms'] = context.create_uniform_buffer(16*MAX_OBJECT_COUNT*4)

...

context.device.buffer_update(descriptors['transforms'].rid,
    0, 16*MAX_OBJECT_COUNT*4, get_transforms())

...

func update_object_transforms(transforms: Array[Transform3D]) -> void:
    object_transforms = transforms

func get_transforms() -> PackedByteArray:
    var fbuf := PackedFloat32Array()
    assert(len(object_transforms) <= MAX_OBJECT_COUNT)
    var t := []
    for i in object_transforms:
        t.append(Projection(i))
    for i in (MAX_OBJECT_COUNT - len(object_transforms)):
        t.append(Projection.IDENTITY)
    for i in MAX_OBJECT_COUNT:
        fbuf.append_array([ t[i].x[0], t[i].x[1], -t[i].x[2], -t[i].x[3],
                           t[i].y[0], t[i].y[1], -t[i].y[2], -t[i].y[3],
                           -t[i].z[0], -t[i].z[1], t[i].z[2], t[i].z[3],
                           -t[i].w[0], -t[i].w[1], t[i].w[2], t[i].w[3]])

    var bytebuf := PackedByteArray()
    bytebuf.resize(4 * fbuf.size())
    bytebuf.fill(0)
    for i in range(len(fbuf)):
        bytebuf.encode_float(i*4, fbuf[i])
    return bytebuf

```

Amit pedig a shaderben fel tudunk használni:

```

// same as in gaussian_splatting_rasterizer.gd
#define MAX_OBJECT_COUNT 16

...

struct Splat {
    vec3 position;
    float time;
    float covariance[6]; // Contains top triangle of symmetric matrix
    float opacity;
    float id;
    float sh_coefficients[16*3]; // Spherical harmonic coefficients in increasing order
};

...

layout (std140, set = 0, binding = 7) restrict uniform Transforms {
    mat4 transforms[MAX_OBJECT_COUNT];
};

...

```

```

// --- FRUSTUM CULLING ---
vec3 splat_pos = splat.position*model_scale;
vec4 view_pos = view_matrix * transforms[int(splat.id + 0.5)] * vec4(splat_pos, 1.0);
vec4 clip_pos = projection_matrix * view_pos;

...

mat3 curr_transform = mat3(transforms[int(splat.id + 0.5)]);
mat3 cov_mx = curr_transform * DECODE_COVARIANCE(splat.covariance) * transpose(curr_transform);
const vec3 covariance = project_covariance(cov_mx, splat_scale, view_pos.xyz, dims);

...

```

Automata objektum-megtalálás

Jelenleg nem támogatott új objektumok létrehozása illetve eltüntetése, láthatóvá illetve láthatatlanná tétele futás-időben. Támogatott viszont az, hogy egy, a Godot Node-ok között is megtalálható objektumot a SceneTree-hez hozzáadva automatán be legyen töltve a splat-os objektum, és a megfelelő Node transzformációja legyen rá hattatva, figyelembe véve a szülő node-ok transzformációját is, így lehetséges valós időben mozgatni, transzformálni őket ugyanúgy, mint a sima modelleket.

A következő script a `class_name` segítségével meg fog jelenni az editorban, mint hozzáadható Node, és lesz egy állítható paramétere, a `ply_file`. Ide be lehet állítani a megfelelő path-t.

```

class_name SplatMesh extends Node3D

@export var ply_file: String

func is_splat_mesh():
    pass

Ezután a gyökérnode induláskor összeszedi az objektumokat, és eltárolja őket, hogy a transzformációikat el tudja küldeni:

var splat_meshes : Array[SplatMesh] = []

func _ready() -> void:
    find_by_method(self, StringName("is_splat_mesh"), splat_meshes)
    assert(len(splat_meshes) <= GaussianSplattingRasterizer.MAX_OBJECT_COUNT)
    var splat_filenames := []
    for m in splat_meshes:
        splat_filenames.append(m.ply_file)

    init_rasterizer(splat_filenames)
    ...

# source: https://forum.godotengine.org/t/how-do-you-get-all-nodes-of-a-certain-class/9143
func find_by_method(node: Node, method_name : StringName, result : Array) -> void:
    if node.has_method(method_name) and node.is_visible_in_tree():
        result.push_back(node)
    for child in node.get_children():
        find_by_method(child, method_name, result)

```

Az összeszedéshez biztosan van jobb módszer, mint a tagfüggvénynév alapján való keresés, de én limitált keresés során nem találtam, és ez működik. Típuslekérdezés csak beépített típusokra ad eltérő eredményt.

```

func _process(delta: float) -> void:
    ...
    var splat_transforms : Array[Transform3D] = []
    for m in splat_meshes:

```



```
splat_transforms.append(m.global_transform)
rasterizer.update_object_transforms(splat_transforms)
```

Fontos megemlíteni a szálakat, ugyanis a rasterizer függvényeit alapvetően a render thread-en hívjuk, ez az objektum-transzformációk feltöltése viszont, a kameratranszformációk lekérdezéséhez hasonlóan a "sima" thread-en történik.

Mélységhelyes összekombinálás

Ahhoz, hogy a képen össze lehessen fésülni a pontfelhős, és a sima megjelenítést, amelyek teljesen külön pipeline-on futnak, szükséges a pontfelhős megjelenítésnél is, a végső renderelt képen egy pixelenkénti mélységérték szerzése. Ennek segítségével legalább pixelenként el lehet dönteni, hogy a két renderelt kép közül melyik nyerjen. A sima pipeline-nál már eléri a játékmotor a mélységet, nekünk a splat-os megjelenítésnél kell ezt elérhetővé tennünk:

Projection:

```
data.pos_z = (ndc_pos.z + 1.0)*0.5;
```

Render:

```
shared float[WORKGROUP_SIZE] pos_z_tile;
```

```
...
```

```
for (uint i = 0; i < num_iterations && shared_t > MIN_FACTOR; ++i) {
```

```
...
```

```
    for (uint j = 0; j < chunk_size && t > MIN_ALPHA; ++j) {
```

```
        ...
```

```
        float pos_z = pos_z_tile[j];
```

```
        ...
```

```
        if (alpha > 0.2) {
```

```
            weighted_depth += (1.0 - pos_z) * alpha * t;
```

```
            total_weight += alpha * t;
```

```
        }
```

```
        ...
```

```
        t *= (1.0 - alpha);
```

```
    }
```

```
    ...
```

```
}
```

```
vec3 heatmap_color = mix(vec3(0,0,1), vec3(1,0.2,0.2), num_splats*5e-4) * (1.0 - t) * heatmap_factor;
```

```
float final_depth = total_weight > 0.0 ? (weighted_depth / total_weight) : 0.0;
```

```
imageStore(rasterized_image, ivec2(image_pos), vec4(blended_color + heatmap_color, final_depth));
```

```
...
```

Main spatial shader:

```
void fragment() {
```

```
    ALBEDO = srgb_to_linear(texture(render_texture, SCREEN_UV).rgb);
```

```
    DEPTH = texture(render_texture, SCREEN_UV).a;
```

```
}
```

Projektek kombinálása

A splat-os és az autós Godot projektek összefésülése egészen könnyen ment, a fájlok összemásolásán túl a `project.godot` fájlt kellett kompatibilissá tenni mindkét projekt részére, többek között a definiált (billentyűzet) bemenetet, és a globális scripteknek megfelelő scripteket (autoload) kellett átemelni. Szerencsére a `project.godot` egy szöveges fájlformátum, sima config fájlként viselkedik, így akár kézzel is könnyű volt az egyes bejegyzéseket átemelni, a maradék beállítást pedig könnyen megtaláltam az editorban.

Továbbá mindkét projektben volt az alapértelmezett jelenet, aminek a node-jaihoz a szükséges scriptek hozzá voltak adva. Kis logikázás után úgy láttam, hogy a splat-os projekt gyökérnode-ja kell, hogy maradjon, többek között

azért is, hogy a hozzáadott SplatMesh-eket megtalálja a jelenetben. Nem volt különösebb gond az autós projekttel, ha annak a gyökérnode-ját a splat-os projekt gyökérnode-ja gyerekeként tettem be.

Ami következményként nehézséget okozott, az a stuttering kiküszöbölése, amit az okozott, hogy az autós projektben a kamerát egy script állította, míg a splat-os projektben ugyanezen (aktív) kamera transzformációját egy script továbbította a GPU-nak, és ez a két script valamiért nem meghatározott sorrendben futott. A process priority állításával a probléma megoldódott.

Adatok beolvasása

A bemeneti adatformátum a .ply, illetve ennek egy speciális esete. Eredetileg a megjelenítő projekt csak olyan modelleket tudott megjeleníteni, amiknek a 62 property-je mind szerepel, és egy konkrét sorrendben van:

```
x, y, z, nx, ny, nz, f_dc_0, f_dc_1, f_dc_2, f_rest_0, f_rest_1, f_rest_2, f_rest_3, f_rest_4,
f_rest_5, f_rest_6, f_rest_7, f_rest_8, f_rest_9, f_rest_10, f_rest_11, f_rest_12, f_rest_13,
f_rest_14, f_rest_15, f_rest_16, f_rest_17, f_rest_18, f_rest_19, f_rest_20, f_rest_21, f_rest_22,
f_rest_23, f_rest_24, f_rest_25, f_rest_26, f_rest_27, f_rest_28, f_rest_29, f_rest_30, f_rest_31,
f_rest_32, f_rest_33, f_rest_34, f_rest_35, f_rest_36, f_rest_37, f_rest_38, f_rest_39, f_rest_40,
f_rest_41, f_rest_42, f_rest_43, f_rest_44, opacity, scale_0, scale_1, scale_2, rot_0, rot_1,
rot_2, rot_3
```

Ez akkor ütközött problémába, amikor egy másik forrásból származó modellt szerettem volna megjeleníteni, ami .splat formátumban volt. Addig nem probléma, hogy az online [Supersplat](#) segítségével át lehet konvertálni ezt .ply fájlra, viszont ebből a modelltől hiányoztak bizonyos property-k, 14 volt összesen, és a meglevők sem megfelelő sorrendben voltak:

```
x, y, z, opacity, rot_0, rot_1, rot_2, rot_3, f_dc_0, f_dc_1, f_dc_2, scale_0, scale_1, scale_2
```

Így módosítottam a beolvasó részt, hogy 0-ként olvassa a hiányzó adatokat, illetve a megfelelő pozícióba tegye a property-ket. Ez azért működik, mert ami hiányzik, az a 45 db spherical harmonikusokhoz szükséges érték, amiket lehet 0-ra inicializálni, illetve a normálvektor, amit pedig nem használ fel a beolvasó.

```
func parse(path : String) -> void:
    var file := FileAccess.open(path, FileAccess.READ)
    var line := file.get_line().split(' ')
    while not line[0] == 'end_header':
        line = file.get_line().split(' ')
        match line[0]:
            'format': file.big_endian = line[1] == 'binary_big_endian'
            'element': size = int(line[2])
            'property': properties.push_back(line[2])
    vertices = file.get_buffer(size*len(properties) * 4).to_float32_array()
    if properties.hash() != DEFAULT_PROPERTIES.hash():
        var prop_inverse := {}
        for i in properties.size():
            prop_inverse[properties[i]] = i
        var new_vertices := PackedFloat32Array()
        new_vertices.resize(size*DEFAULT_PROP_CNT)
        for i in size:
            for pi in DEFAULT_PROP_CNT:
                new_vertices[i * DEFAULT_PROP_CNT + pi] =
                    vertices[i * len(properties) + prop_inverse[DEFAULT_PROPERTIES[pi]]]
                if DEFAULT_PROPERTIES[pi] in prop_inverse else 0
    properties = DEFAULT_PROPERTIES.duplicate()
    vertices = new_vertices
```

Eredmények, összefoglalás

Sikeresen megjelenítettem több autót, amiket rá tudtam húzni egy már meglevő fizikai szimulációra, és össze tudtam ezt kombinálni egy sima módon megjelenített pályával. Sokat tanultam abból a szempontból is, hogy hogyan lehet egy meglevő kódot kicsit módosítani úgy, hogy elérjem azt a módosulatot, amit szeretnék. A fél éves projektet sikeresnek titulálom.

Fejlesztési lehetőségek

A továbbiakban a teljesség igénye nélkül az alábbi fejlesztési lehetőségek végrehajtása lenne előnyös:

- jobb editor-integráció, és könnyebben használhatóvá tenni
- autoload-ba tenni a Gsplat rendereléshez szükséges komponenseket
- objektumok láthatóságának állítása futásidőben, hozzáadás, levétel
- publikálni az AssetLib-ben
- SplatMesh push-olja a transzformját a rasterizernak ahelyett, hogy a main szedi össze
- jobb mélység-kombinálás splat és sima mesh között
- .ply beolvasott fájl hiányzó property-jeinek 0-kkal feltöltése helyett nem feltölteni a gpu-ra, illetve ahol nem a 0 a helyes, ott is megfelelő érték
- konzultálni a .ply fájlformátummal, hogy esetlegesen másmilyen, de helyes fájlokat is be tudjon olvasni
- .splat fájl beolvasásának képessége
- újralightolható .ply fájlok kezelése